DART: Draft-Agreement Routing for Training-Free Adaptive Thinking Budgets in Hybrid Reasoning Models

Jungseob Lee[1] **Seongtae Hong**[1] **Seungjun Lee**[2] **Jaehyung Seo**[3] **Junyoung Son**[1]
Sugyeong Eo[4] **Chanjun Park**[5] **Hyeongju Park**[6] **Hyeonseok Moon**[1*] **Heuiseok Lim**[1*]
[1]Korea University [2]42dot [3]Konkuk University [4]Yonsei University
[5]Soongsil University [6]Kumoh National Institute of Technology
{omanma1928, ghdchlwls123, s0ny, glee889, limhseok}@korea.ac.kr
seungjun.lee@42dot.ai, seojae777@konkuk.ac.kr, s.eo@yonsei.ac.kr
chanjun.park@ssu.ac.kr, hyungju1203@gmail.com

arXiv:2606.23181v4 [cs.AI] 3 Oct 2026

Abstract

Hybrid reasoning models can answer directly or spend extra tokens on extended thinking. A practical router should choose between these modes for each query, so easy problems avoid unnecessary reasoning and hard problems receive enough budget to finish the answer. Existing routers move in this direction, but they typically require labeled training data or fix thinking budgets up front, ignoring answer-level evidence from the model itself. We introduce DART, a training-free routing framework that samples two cheap no-think drafts, accepts direct answering when the drafts agree, and predicts a thinking budget from draft entropy when they disagree. Across the main comparisons, DART preserves or improves always-thinking accuracy in most settings while reducing thinking-token use. Accuracy improves by up to +9.0 points on Olympiad-level math and by up to +22.5 points on code under execution-based equivalence, while thinking-token use drops by 32–73%. The Stage 1 signal extends across model scales (0.6B–32B), model families, and API-only hosted settings, with no labeled data and no gradient updates required. Our code is available at https://github.com/js-lee-AI/DART.

1 Introduction

Recent reasoning-capable large language models (LLMs) increasingly expose hybrid inference interfaces across open-weight models (Qwen Team, 2025; DeepSeek-AI, 2025; DeepSeek-AI et al., 2025; Google DeepMind, 2026; Yu et al., 2025; NVIDIA, 2026) and hosted services (OpenAI, 2026; Anthropic, 2026). These systems can invoke an extended-reasoning mode (THINK), answer directly (NOTHINK), and, in some cases, expose controls over the reasoning budget. The appeal is accuracy and controllability. THINK mode can substantially improve task accuracy on math, code, and competition benchmarks (Snell et al., 2025; Feng et al., 2025), while budget control lets a deployment assign smaller or larger reasoning budgets according to query difficulty. Together, mode selection and budget allocation let a deployment choose both whether to think and how much reasoning to spend for each query.

However, thinking is not free. For many easy queries the model commits thousands of tokens to deliberate over a problem where a direct response would have sufficed, and recent studies report that extended reasoning sometimes degrades accuracy below the no-think baseline on the easier slice of the same benchmark (Ding et al., 2025; Sui et al., 2025). Running every query in THINK mode is impractical in deployment. Easy queries still pay a substantial thinking cost (Alomrani et al., 2025), while on harder queries the model can exhaust the single-call length limit during the reasoning trace before emitting a complete final answer, unless the deployment provisions a very large token allowance. Provisioning such allowances and the corresponding compute for broad deployment is operationally expensive, making fixed always-thinking deployment difficult to sustain.

Adaptive reasoning routers have emerged to address this overhead by choosing an inference strategy separately for each query, such as direct answering, extended thinking, or assigning a query-specific thinking budget (Li et al., 2025; Pan et al., 2025). Existing strategies either train a classifier or fine-tune a policy, or rely on confidence and entropy heuristics (Kuhn et al., 2023; Kadavath et al., 2022). This leaves two practical problems. First, trained routers need labeled difficulty data and model-specific retraining whenever the target model changes, which limits their use in API-only deployments. Heuristic scores avoid retraining but remain indirect proxies for the binary choice between THINK and NOTHINK mode. Second, com-

*Corresponding authors.

mon single-call evaluations place the reasoning trace and final answer under one generation cap. Under tight caps, the model can exhaust the budget during reasoning before emitting a complete final answer, so comparing routers against this truncated always-thinking baseline measures a protocol artifact rather than always-thinking accuracy with an intact answer span. Appendix B quantifies this artifact directly.

In this paper, we introduce DART (**D**raft-**A**greement **R**outing for **T**hinking), a training-free two-stage router for hybrid reasoning models. DART uses cheap NoThink drafts as a query-level difficulty probe. Stage 1 accepts a unanimous draft answer under a pluggable equivalence function and routes only disagreement cases to Think mode, while Stage 2 maps draft entropy to a query-specific thinking budget so harder routed queries receive more reasoning and easier routed queries stop earlier. The pipeline requires no labeled difficulty data or gradient updates and separates answer generation from the thinking trace to avoid single-call truncation artifacts. Across the evaluations below, DART reduces thinking-token use without giving up the always-thinking accuracy target.

Our contributions are as follows: (a) we characterize draft unanimity as a binary difficulty proxy for hybrid reasoning, with a consistently strong correlation to always-thinking correctness. (b) we propose DART, a training-free router whose Stage 1 draft-agreement decision uses only generated draft answers under a pluggable equivalence function, making it compatible with text-only API access to closed hybrid models, while Stage 2 optionally adds entropy-predicted budgets when token log-probabilities and budget controls are available. (c) in observed point estimates, DART matches or exceeds always-thinking across Qwen3 8B–32B and DeepSeek-V3.2, improving accuracy on both math and code while using 32–73% fewer thinking tokens. (d) the supervised-router diagnostic shows that draft unanimity is more informative than entropy or draft length, reinforcing the Stage 1 signal while DART obtains it without labels or model-specific retraining.

2 Draft-Agreement Routing for Thinking

DART has two stages. SC-Route (§2.2) samples K drafts generated in NoThink mode and accepts the draft answer only when their extracted answers agree. Stage 2 (§2.3) allocates a query-specific thinking budget to disagreement cases from draft entropy, concentrating tokens on harder queries. Figure 1 gives the pipeline overview and Algorithm 1 the full inference procedure.

2.1 Preliminaries and Objective

Let q be an input query and $\mathcal{M}$ a system exposing two inference modes, Think (extended chain-of-thought) and NoThink (direct response). Given a query distribution where some queries admit a correct answer without thinking and others require thinking, our goal is a routing policy $\pi : q \mapsto \{\text{NoThink}, \text{Think}\}$ that minimizes expected thinking-token cost subject to an accuracy constraint, *without training data or gradient updates*. This formulation covers both local models exposing a thinking toggle and API-mediated services that route between thinking and direct-answer endpoints.

2.2 Self-Consistency Routing

Stage 1, SC-Route (Self-Consistency Routing), adapts the agreement signal used in self-consistency decoding (Wang et al., 2023; Aggarwal et al., 2023) to routing. When K drafts generated in NoThink mode produce the same normalized answer, the model has usually resolved the query without needing an explicit thinking trace. The routing decision uses two task-facing modules. The answer extractor $a(\cdot)$ maps each draft to the answer object used by the benchmark, and the equivalence function $\text{eq} : \mathcal{A} \times \mathcal{A} \to \{0, 1\}$ checks whether two extracted answers are equivalent. The method has one hyperparameter (K, the number of drafts) and no thresholds.

Draft. Sample K independent completions in NoThink mode at temperature T.

$$r_1, \ldots, r_K \sim \mathcal{M}(q, \text{ think=false}, T). \quad (1)$$

Route. Compute $a(r_k)$ for each draft. If all K extracted answers are pairwise equivalent under eq (*unanimous agreement*), accept the draft answer. Otherwise, route the query to Think mode.

$$\hat{y} = \begin{cases} a(r_1), & \text{if drafts agree under eq,} \\ a(\mathcal{M}(q, \text{Think})), & \text{otherwise.} \end{cases} \quad (2)$$

Design choices. We use strict unanimity at $K{=}2$ rather than majority voting because agreement between two independent stochastic samples concentrates probability $\geq p^2$ on the model's dominant

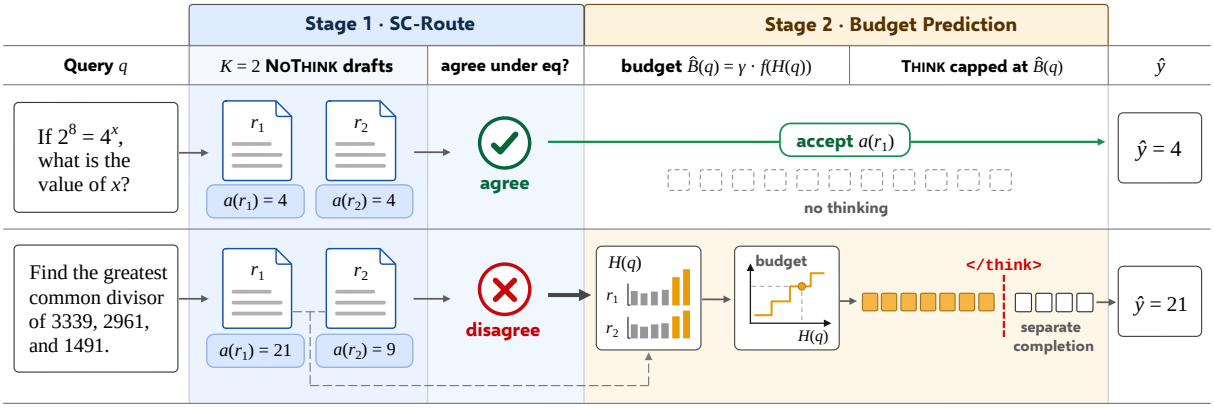


Figure 1: Overview of DART. Stage 1 draws K=2 no-think drafts and accepts the unanimous answer when they agree. On disagreement, Stage 2 maps draft entropy to a query-specific thinking budget and produces the answer in a separate completion.

Algorithm 1 DART Inference Pipeline.

Require: Query q, model $\mathcal{M}$, drafts K, temperature T
Ensure: Response $\hat{y}$
Draw $r_1, \ldots, r_K \sim \mathcal{M}(q,\ \text{think=false},\ T)$
$a_i \leftarrow \textsc{ExtractAnswer}(r_i)$ for $i = 1, \ldots, K$
if all a_i agree under eq **then**
  return a_1
else
  $\hat{B} \leftarrow \gamma \cdot f(H(q))$
  $r^* \leftarrow \mathcal{M}(q,\ \text{think=true},\ B{=}\hat{B})$
  return $\textsc{ExtractAnswer}(r^*)$
end if

answer, while the analysis below shows that this low-cost rule keeps accepted-answer precision high without thresholds and avoids the larger K cost of self-consistency.

Pluggable equivalence. The equivalence module eq is the only domain-specific component. It can be string normalization for math, sandboxed execution match for code, structured-output comparison, and so on. The routing mechanism itself is unchanged across domains. See Appendix E.2 for normalization rules and the code execution protocol.

2.3 Budget Prediction and Two-Stage Generation

Stage 2 is an evaluation-time protocol applied only to queries Stage 1 routes to THINK mode.

Motivation. Disagreement queries vary in how much thinking they need. A fixed budget wastes tokens on the easier slice and truncates the harder slice. We adopt draft entropy as the uncertainty signal for a query-specific budget. Higher draft entropy maps to a larger thinking budget, while lower-entropy queries are served at tight budgets with the final answer produced in a separate completion call. This two-stage design avoids the one-phase measurement artifact where the answer is silently truncated under tight thinking budgets.

Letting $p_t^{(k)}(v) = p_\theta\big(v \mid q, y_{<t}^{(k)}\big)$ denote the model's next-token distribution at position t of draft k, draft entropy is the mean token-level entropy over all K drafts.

$$H(q) = -\frac{1}{K}\sum_{k=1}^{K}\frac{1}{|\mathcal{T}_k|}\sum_{t\in\mathcal{T}_k}\sum_{v\in V} p_t^{(k)}(v)\log p_t^{(k)}(v). \tag{3}$$

We map entropy to budget via accuracy-label-free isotonic regression (Ayer et al., 1955) f on a separate probe run of disagreement-flagged math queries, and apply a safety margin γ=1.5 to keep the predicted budget above the actual thinking-token cost at the matched entropy level.

$$\hat{B}(q) = \gamma \cdot f\big(H(q)\big). \tag{4}$$

For models exposing a stop-thinking control, such as Qwen3, when the thinking trace reaches $\hat{B}(q)$, we inject a `</think>` token and generate the answer in a separate completion call. The probe run, the isotonic-regression fit, and the choice of γ=1.5 are detailed in Appendix F.

3 Experimental Setup

3.1 Models

We evaluate two hybrid reasoning model families, Qwen3 (8B, 14B, 32B) (Qwen Team, 2025) and DeepSeek-V3.2 (DeepSeek-AI et al., 2025) via the hosted DeepSeek API. The evaluated interfaces expose explicit THINK and NOTHINK controls through chat-template controls or API endpoints,

Model	Benchmark	Accuracy (%)				Efficiency	
		NT	AT	SC-Route	**DART** (vs AT)	Think↓	Route%
Qwen3-8B	MATH-500	76.6	85.6	87.6	**88.2** (+2.6)	67%	78.0
	OlympiadBench	49.8	**71.5**	70.4	69.8 (−1.7)	45%	52.1
	HumanEval	60.4	59.1	78.7	**78.7** (+19.6)	55%	54.9
	MBPP	59.5	64.2	67.5	**68.9** (+4.7)	46%	57.6
Qwen3-14B	MATH-500	81.2	87.6	87.6	**87.6** (+0.0)	73%	83.4
	OlympiadBench	51.5	53.0	62.0	**62.0** (+9.0)	51%	58.0
	HumanEval	71.3	66.5	78.7	**78.7** (+12.2)	60%	68.9
	MBPP	64.6	64.6	68.1	**68.1** (+3.5)	51%	63.0
Qwen3-32B	MATH-500	82.2	86.2	88.5	**88.5** (+2.3)	69%	80.0
	OlympiadBench	50.5	54.0	58.5	**58.5** (+4.5)	52%	59.5
	HumanEval	79.9	72.6	95.1	**95.1** (+22.5)	63%	76.8
	MBPP	65.8	65.8	71.2	**71.2** (+5.4)	51%	63.0
DeepSeek-V3.2*	MATH-500	84.8	88.4	90.6	**92.6** (+4.2)	56%	85.0
	OlympiadBench	63.6	66.1	67.1	**69.1** (+3.0)	32%	60.1

Table 1: Accuracy (in %) and thinking-token efficiency on Qwen3-8B/14B/32B and DeepSeek-V3.2. Accuracy columns report NT, AT, SC-Route (Stage 1 routing only, with disagreement queries falling back to AT), and the full DART pipeline (with vs-AT delta in green/red). Efficiency columns report Think↓, the thinking-token reduction vs AT, and Route%, the rate at which Stage 1 accepts unanimous drafts. Shading marks DART matching or exceeding AT, and bold marks the better of DART and AT in each row. *DeepSeek-V3.2 results use the hosted API.

which we treat as the routing primitive. The same routing pipeline applies to all backbones with no model-specific tuning, and we use a single shared hyperparameter setting across model families.

3.2 Benchmarks

We evaluate on five public benchmarks spanning mathematical reasoning, code generation, and competition math, including **MATH-500** (Hendrycks et al., 2021; Lightman et al., 2024) (500 competition-level problems), **OlympiadBench** (He et al., 2024) (572 Olympiad-level problems), **AIME 2024/2025** (Jia, 2024; Organization, 2025) (30 problems each), **HumanEval** (Chen et al., 2021) (164 code generation problems), and **MBPP** (Austin et al., 2021) (257 code generation problems from the sanitized test split).

Splits. Code routing executes both drafts against each problem's reference tests, so no code problems or tests are held out and DART rows report all HumanEval and MBPP problems. The Stage 2 entropy-to-budget map is fit label-free on a separate 100-problem probe run, 71 of whose problems also appear in the MATH-500 evaluation set.

3.3 Baselines and Metrics

Baselines. Our **always-thinking (AT)** baseline uses a two-phase protocol that emits the thinking trace and the final answer in separate completion calls, eliminating the answer truncation that affects the conventional single-budget one-phase protocol (reported only as a measurement-artifact diagnostic in Appendix B). **No-think (NT)** fixes the thinking switch off for all queries. Additional baselines comprise majority voting (MVk), paper-defined self-prompted routing baselines (MC-Binary, MC-Conf), and supervised routers (MLP, gradient-boosted trees) trained on labels with draft entropy, draft length, and optional unanimity features.

Metrics. We report accuracy, accept precision, and the point-biserial correlation r_{pb} between draft unanimity and AT correctness. Two efficiency metrics complement accuracy. **Think↓** measures the reduction in mean emitted thinking tokens vs. AT across all queries, counting actually emitted tokens rather than the predicted budget. **Route%** is the unanimous-accept rate, namely the fraction of queries served by Stage 1 with zero thinking tokens.

Evaluation protocol details are given in §E.1, and supervised-router CV setup, software versions, sampling temperatures, and hardware in Appendix E.

Method	MATH-500	OlympiadBench	HumanEval
Training-free — Single-mode baselines			
NT	76.6 (71%)	49.8 (64%)	60.4 (98%)
AT	85.6 (0%)	**71.5** (0%)	59.1 (0%)
Training-free — NT aggregation			
MV (K=3)	82.4 (13%)	53.7 (−7%)	10.4 (95%)
MV (K=5)	83.4 (−45%)	58.6 (−79%)	10.4 (91%)
MV (K=7)	84.8 (−103%)	58.7 (−151%)	9.8 (88%)
Training-free — Self-prompted routing			
MC-Binary	85.8 (4%)	69.9 (0%)	10.4 (−7%)
MC-Conf	85.8 (24%)	54.9 (25%)	8.5 (44%)
Training-free — Draft-agreement (ours)			
DART	**88.2** (35%)	69.8 (9%)	**78.7** (52%)
Supervised routers (5-fold CV on matched benchmark subsets)			
MLP	81.0 (69%)	58.2 (64%)	62.5 (98%)
GBT	78.8 (64%)	54.0 (51%)	63.7 (88%)
Supv. (ent.+len.)	68.4 (69%)	50.4 (64%)	63.1 (98%)
Supv. +unanimity	84.0 (63%)	50.0 (50%)	63.1 (90%)

Table 2: Routing strategy comparison on Qwen3-8B. Each cell shows accuracy (%) and, in parentheses, the mean total-token saving vs AT averaged over all queries (% reduction in drafts + thinking + answer tokens). Negative values indicate the method generates more total tokens than AT.

4 Results

4.1 Main Results

Table 1 summarizes the main comparison across Qwen3 8B, 14B, and 32B on MATH-500, OlympiadBench, HumanEval, and MBPP, together with DeepSeek-V3.2 on MATH-500 and OlympiadBench. Across these observed point estimates, DART reaches the always-thinking accuracy target while reducing emitted thinking tokens on both open-weight Qwen3 models and the hosted DeepSeek-V3.2 API. It matches or exceeds AT on 13 of 14 model–benchmark pairs and reduces thinking-token use by 32–73%. The single miss is Qwen3-8B on OlympiadBench, where DART trails AT by 1.7 points while still reducing thinking tokens by 45%. AIME 2024/2025 results are reported as exploratory evidence in Appendix A.

Math tasks. On math, DART improves over AT by up to +9.0 points while reducing thinking-token use by 32–73%. The gains appear on six of eight math model–benchmark pairs, including both DeepSeek-V3.2 rows under the same unanimity rule and without model-specific re-tuning. This transfer matters because DeepSeek-V3.2 is evaluated through a hosted API, so the result tests the router beyond locally hosted Qwen3 checkpoints.

Coding tasks. On code, DART improves over AT by up to +22.5 points while reducing thinking-token use by 46–63%. The gains are consistent on HumanEval and MBPP across all three Qwen3 scales. The large HumanEval gains arise because AT underperforms NT on short-form code, and Stage 1 keeps high-agreement code queries on the direct-answer path.

Benchmark	NT	AT	DART	McNemar p	
	accuracy [95% CI]			vs NT	vs AT
MATH-500	76.6 [72.8, 80.2]	85.6 [82.4, 88.6]	**88.2** [85.2, 91.0]	$< .001$	.066
OlympiadBench	49.8 [45.8, 53.8]	**71.5** [67.7, 75.2]	69.8 [65.9, 73.4]	$< .001$	.30

Table 3: Bootstrap 95% confidence intervals and McNemar tests for Qwen3-8B, computed from the main runs.

Statistical reliability. Table 3 reports bootstrap 95% confidence intervals and McNemar tests computed from the problem-level logs of the Qwen3-8B runs in Table 1. The gain of DART over NT is decisive on both benchmarks ($p < .001$). Against AT, DART is statistically indistinguishable, nominally higher on MATH-500 and nominally lower on OlympiadBench, while cutting thinking tokens by 45–67%, which is the accuracy-at-lower-cost claim the paper makes.

4.2 Routing Strategy Comparison

Table 2 compares DART against training-free and supervised baselines on Qwen3-8B. MVk samples multiple NoThink drafts and returns a majority answer, so it aggregates direct answers rather than deciding when to think. MC-Binary asks the model whether thinking is needed, and MC-Conf routes from a self-reported confidence score. The supervised rows train MLP or gradient-boosted tree routers with 5-fold cross-validation, using draft entropy, mean draft length, and, where indicated, Stage-1 unanimity. HumanEval supervised rows use execution-based labels, and DART HumanEval is reported on the full benchmark.

Training-free baselines. MVk (K=3–7) plateaus below AT on math and collapses on HumanEval to below 11%. The self-prompted MC-Binary row reaches 85.8% on MATH-500 but also collapses on HumanEval to 10.4%. Token-level vote aggregation and self-prompted confidence both fail on code where draft-level execution agreement succeeds, suggesting agreement captures a domain-agnostic difficulty signal the others miss. The MVk rows instantiate NoThink parallel scaling (Ma et al., 2025), whose vote saturates at 84.8 by K=7 on MATH-500. Difficulty-Adaptive Self-Consistency (Wang et al., 2024a), instantiated over the same drafts with a confidence-based

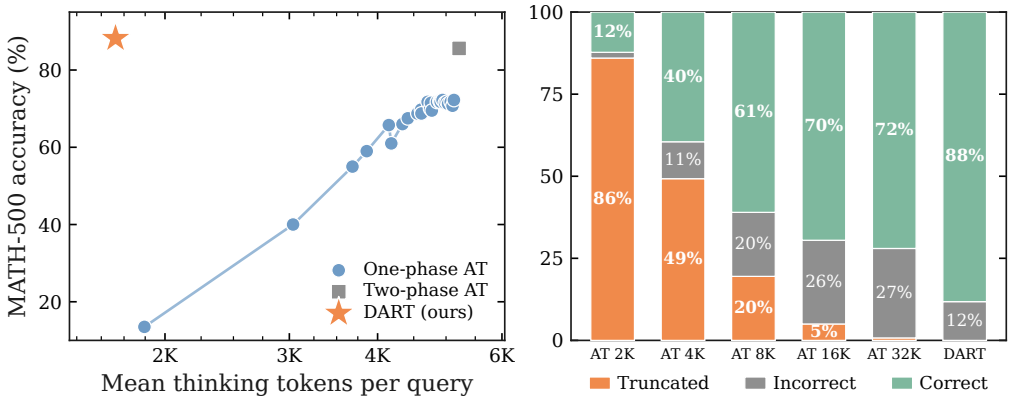


Figure 2: Efficiency Pareto (left) and response-level failure-mode breakdown (right) for Qwen3-8B on MATH-500. Both panels place each method at the thinking tokens it actually emits. The one-phase sweep uses the diagnostic grader of §E.1, while the two-phase AT and DART points use the main protocol of Table 1.

stopping rule, returns the identical majority answer on all 500 MATH-500 problems while drawing 4.9 samples on average instead of seven, so it lowers sampling cost without lifting that ceiling. Neither verifier-free aggregation reaches DART's 88.2, because majority voting over NoThink drafts never makes the think-or-not decision.

Supervised routers. Even with 250 labels, gradient-boosted trees (78.8%) and MLPs (81.0%) underperform training-free DART (88.2% on MATH-500). A supervised router augmented with the draft-unanimity feature improves to 84.0%, but still trails training-free DART. Its learned unanimity coefficient (-4.43) is much larger than entropy's (-0.13), showing that the diagnostic supervised model rediscovers the same signal that DART uses directly without labels. Without the unanimity feature, supervised routers learn a degenerate "always NT" policy.

4.3 Token and Latency Efficiency

Token budget. DART reduces generated tokens while improving accuracy over AT. Table 4 reports mean generation tokens on Qwen3-8B against the same two-phase AT baseline as Table 1, counting drafts, thinking traces, and answers on MATH-500 and thinking traces alone on HumanEval.

On MATH-500, DART averages 3.5K total generated tokens compared with 5.4K for AT and gains 2.6 accuracy points, yielding a 35% total-token reduction at positive accuracy delta. On HumanEval, DART reduces thinking tokens by 55% while gaining 19.6 points. The code saving arises because accepted NoThink drafts are short code snippets, whereas AT spends long reasoning traces on many queries that Stage 1 answers directly. Figure 2 complements the table for MATH-500 by visualizing the efficiency frontier and separating false accepts from Think-mode failures for the error analysis in Section 5.2.

Benchmark	AT tok.	DART tok.	Savings	Δ Acc
MATH-500	5,382	3,475	−35%	+2.6
HumanEval	2,980	1,339	−55%	+19.6

Table 4: Mean generation tokens and accuracy delta against the two-phase AT baseline on Qwen3-8B. The MATH-500 row counts drafts, thinking, and answers, the HumanEval row thinking only.

Wall-clock latency. Token savings also reduce wall-clock latency. Table 17 in Appendix H reports that DART averages 29.5 seconds on Qwen3-8B MATH-500, compared with 67.6 seconds for AT. The 2.3-fold speedup comes from the 78% of queries accepted after parallel NoThink drafts, which finish in 14.6 seconds on average and avoid a Think pass.

5 Analysis

5.1 Difficulty Alignment and Generalization

This analysis tests the assumption behind Stage 1. Draft agreement should mark queries that can be answered directly, while disagreement should identify cases that need Think mode. Under this assumption, Stage 1 should accept fewer examples as benchmark difficulty rises while keeping accepted-answer precision high. The signal should also remain positive across model families and scales.

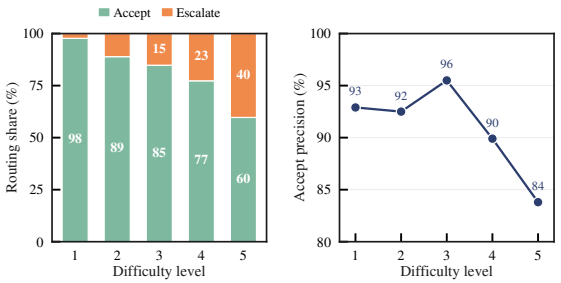


Figure 3: Difficulty-stratified Stage-1 behavior on MATH-500 (Qwen3-8B). The left panel shows the no-think accept share and the share escalated to thinking across difficulty levels 1–5. The right panel shows precision among accepted no-think answers at each level.

Difficulty alignment. Stage 1 accepts fewer queries as annotated difficulty increases while preserving high precision among accepted answers. Figure 3 reports MATH-500 difficulty levels for Qwen3-8B. The Stage 1 accept rate decreases monotonically from 97.7% at Level 1 to 59.7% at Level 5, while accepted-answer precision remains between 83.8% and 95.5% across all five levels. This pattern supports using agreement as the Stage 1 gate because it identifies the subset where direct NoThink answers are reliable and sends the remaining harder problems to Think mode. Aggregated over all queries, accepted answers are 90.8% correct on MATH-500 and 81.9% on OlympiadBench, 14 and 32 points above the unconditional NoThink accuracies of 76.6 and 49.8 in Table 1, so the accept-precision-versus-base-rate gap measures the gate's strength directly.

Robustness across model families. Using the model–benchmark settings summarized in Table 1 and the Qwen3 size sweep in Table 5, draft agreement remains positively associated with AT correctness across every tested pair. The point-biserial correlation r_{pb} is positive and significant ($p < 0.001$) across two model lineages, two architectures, and scales from 0.6B to 32B. The Qwen3 scale comparison is stable, with $r_{pb} = 0.56$ at 8B and 0.57 at 32B, indicating that the signal is not confined to one parameter scale.

Scale transfer. Token savings persist across the Qwen3 scale sweep, but accuracy parity depends on model capability. Table 5 reports Qwen3 0.6B–32B results for the benchmark rows completed under the matched full-pipeline protocol. DART trails AT by 2.0–7.5 points on MATH-500 at 0.6B–4B. At 8B–32B, it matches or exceeds AT on all scale rows except Qwen3-8B OlympiadBench, the same −1.7 point exception reported in the main comparison, while retaining 45–73% thinking-token reductions. The scale sweep explains when the routing signal is reliable enough to match AT.

Model	BM	NT	AT	DART (vs AT)	Think↓
0.6B	MATH-500	38.5	**56.0**	54.0 (−2.0)	40%
	HumanEval	27.4	48.8	**50.6** (+1.8)	9%
	MBPP	34.6	48.2	**51.4** (+3.2)	16%
1.7B	MATH-500	52.5	**65.5**	58.0 (−7.5)	47%
	HumanEval	55.5	85.4	**86.6** (+1.2)	40%
	MBPP	48.2	57.2	**61.9** (+4.7)	28%
4B	MATH-500	66.5	**70.5**	65.5 (−5.0)	48%
	HumanEval	67.1	74.4	**91.5** (+17.1)	58%
	MBPP	56.4	62.6	**66.9** (+4.3)	41%
8B	MATH-500	76.6	85.6	**88.2** (+2.6)	67%
	OlympiadBench	49.8	**71.5**	69.8 (−1.7)	45%
	HumanEval	60.4	59.1	**78.7** (+19.6)	55%
	MBPP	59.5	64.2	**68.9** (+4.7)	46%
14B	MATH-500	81.2	87.6	**87.6** (+0.0)	73%
	OlympiadBench	51.5	53.0	**62.0** (+9.0)	51%
	HumanEval	71.3	66.5	**78.7** (+12.2)	60%
	MBPP	64.6	64.6	**68.1** (+3.5)	51%
32B	MATH-500	82.2	86.2	**88.5** (+2.3)	69%
	OlympiadBench	50.5	54.0	**58.5** (+4.5)	52%
	HumanEval	79.9	72.6	**95.1** (+22.5)	63%
	MBPP	65.8	65.8	**71.2** (+5.4)	51%

Table 5: DART across Qwen3 model sizes (0.6B–32B).

Routing hyperparameter sensitivity. Table 6 and Appendix C report the K sweep, and Appendix D reports thinking-budget cap and draft-temperature sweeps. Unanimity at K=2 trades accept rate against accept precision more favorably than K=3 majority voting. The budget sweep shows that accuracy plateaus once the cap exceeds the entropy-predicted average, and temperature sensitivity is mild around the chat-template defaults (T=0.6–0.8). Higher T inflates draft disagreement without an accuracy benefit. Appendix G reports a self-verification alternative that detects only 22.2% of wrong NoThink drafts.

Table 6 contrasts unanimity at K=2 and K=3 with K=3 majority voting on MATH-500. Unanimity trades accept rate for accept precision, while majority voting at K=3 accepts every query at substantially lower precision.

Table 14 in Appendix C reports the routing signal on three multiple-choice benchmarks (ARC-Challenge (Clark et al., 2018), MMLU-Pro (Wang et al., 2024b), GPQA-D (Rein et al., 2024)), where the point-biserial correlation between unanimity

K	Decision rule	Accept%	Prec.%
2	Unanimity	78.0	90.8
3	Unanimity	73.6	93.5
3	Majority vote	100.0	82.4

Table 6: Effect of the number of drafts K on routing quality (Qwen3-8B, MATH-500).

and AT correctness weakens sharply (0.13, 0.06, -0.009), unlike the open-ended math/code settings of Table 1. The failure has a structural explanation. For $K{=}2$ drafts from distribution $\mathbf{p} = (p_1, \ldots, p_C)$ over C options, the collision probability is $P(\text{agree}) = \sum_{i=1}^{C} p_i^2$. For a 4-choice MCQ where the model places 60% mass on one option, $P(\text{agree}) \geq 0.36$, so agreement occurs frequently even when the model is wrong, breaking the unanimity precision required by Stage 1.

5.2 Error Analysis

Oracle routing analysis. For the Qwen3-8B MATH-500 setting in Table 1, an oracle routing diagnostic that selects the better mode for each query with ground-truth labels reaches 91.2%. DART reaches 88.2% in the same setting, capturing 79% of the oracle gain over the 76.6% no-think baseline. The remaining 3.0-point gap motivates the response-level error taxonomy below, visualized in the right panel of Figure 2.

Error taxonomy. Of DART's 59 errors on MATH-500, 36 are false accepts (61%), cases where drafts agree on the wrong answer, and 23 are THINK-mode failures (39%). False accepts are characterized by 1.38-fold higher draft entropy and 1.71-fold longer drafts compared to true accepts, suggesting these queries are challenging but not separated by the current unanimity gate. This analysis points to the clearest future improvement, reducing false accepts through confidence calibration or additional draft rounds.

5.3 Measurement Artifacts and Fair Comparison

The one-phase accuracy gap is mostly truncation. Table 7 compares two-stage and one-phase measurement at matched budgets on MATH-500, where the two-stage column runs DART at a fixed budget. One-phase AT reaches only 13.5% at 2K with 86.0% of responses truncated and recovers to 72.0% once truncation falls below 1%, so its accuracy tracks its own truncation rate. We therefore adopt the two-phase protocol for the AT baseline. A broader budget sweep from 2K to 60K is provided in Appendix B.

	Budget	**2-stage**	**1-phase**	Δ	**Trunc%**
	2K	69.75	13.50	+56.25	86.0
	4K	70.00	40.00	+30.00	49.2
	6K	71.25	55.00	+16.25	28.7
	8K	70.00	61.00	+9.00	19.5
	12K	71.75	68.75	+3.00	9.2
	16K	72.25	69.50	+2.75	5.0
	24K	72.25	70.75	+1.50	2.8
	32K	72.25	72.00	+0.25	0.8
about 2.8K (**DART**)		**88.2**	—	—	0.0

Table 7: Matched-budget measurement protocols on MATH-500 (Qwen3-8B). Two-stage emits thinking and answer in separate completions, while one-phase combines them in one budget. Budget rows run DART at a fixed budget under a string-extract grader on 400 problems. The final row is the main-protocol result of Table 1. Trunc% the one-phase truncation rate.

Matched-budget accuracy depends on the measurement protocol. Table 7 reports this comparison at eight budget points from 2K to 32K. The accuracy gap Δ collapses monotonically from 56 points at 2K to 0.25 points at 32K, tracking the one-phase truncation rate almost exactly (Pearson $r{>}0.99$). Thinking capacity is not matched across the two columns, since the one-phase column emits 2.3-fold to 4.4-fold more thinking tokens at every budget. The gap therefore has two sources this table cannot separate, an intact answer span and the 78 to 82 percent of queries Stage 1 answers directly. It does establish that one-phase measurement understates a fixed-budget baseline most severely at tight budgets.

One-phase AT plateaus under increased budget. The extended one-phase budget sweep in Appendix B shows accuracy plateaus near 72% from 28K onward, with truncation rate at or below 1%. Under the main evaluation protocol, two-stage AT reaches 85.6% and DART reaches 88.2% while emitting 1,743 mean thinking tokens against AT's 5,343, the 3.1-fold reduction in Table 8. No fixed one-phase budget matches DART's accuracy under the matched comparison. The same truncation collapse and high-budget plateau replicate on OlympiadBench, showing the pattern on a second math benchmark.

Method	Budget	Acc	Tokens	Reduction
AT two-stage	16K	85.6	5,343	1.0×
NT no-think	—	76.6	0	—
DART	about 2.8K	**88.2**	**1,743**	**3.1**×

Table 8: Iso-accuracy comparison on MATH-500 (Qwen3-8B). Budget is each method's thinking-token allowance, a 16k-token cap for AT and a per-query entropy-predicted value for DART. Tokens is the mean emitted thinking-token count over all 500 queries, and reduction is AT-tokens / DART-tokens. Only 24 of 500 AT queries reach the cap.

Method	Train	Logits	API	Retune
AdaptThink	GRPO	No	✗	Retrain
ThinkSw.	Supervised	No	✗	Retrain
SelfBudg.	Supervised	Yes	✗	Retrain
HBPO	RL	No	✗	Retrain
R2Reason	RL	No	✗	Retrain
DART	**None**	Stage 2*	Stage 1	**None**

Table 9: Training requirements and deployment constraints of adaptive reasoning routers. *Used only for Stage 2 budget prediction.

6 Related Work

Adaptive reasoning and test-time compute. Recent work studies how to allocate test-time computation for reasoning models. Some approaches learn routing or control policies for reasoning depth, including AdaptThink (Zhang et al., 2025a), ThinkSwitcher (Liang et al., 2025), HBPO (Lyu et al., 2025), and Route to Reason (Pan et al., 2025). Others predict the amount of reasoning to spend, such as SelfBudgeter (Li et al., 2025). Recent surveys (Snell et al., 2025; Feng et al., 2025; Alomrani et al., 2025) organize this broader landscape, while related studies examine hybrid-thinking controllability (Wang et al., 2025), over-thinking (Ding et al., 2025; Sui et al., 2025; Lee et al., 2026e), and adaptive mode switching (Zhang et al., 2025b). Table 9 summarizes deployment constraints for the router-style methods most closely related to our setting.

Self-consistency and confidence proxies. Self-consistency (Wang et al., 2023) samples multiple chains and aggregates by majority vote. Adaptive Consistency (Aggarwal et al., 2023) dynamically adjusts the sample count based on running agreement. Semantic entropy (Kuhn et al., 2023) clusters paraphrastically equivalent answers and treats the resulting distribution as a confidence proxy. Universal Self-Consistency (Chen et al., 2024b) extends this idea to free-form generation through an LLM-judged equivalence test. These methods establish agreement and answer dispersion as useful signals for generation-time uncertainty (Lee et al., 2026a,b).

Routing across models or computation paths. Routing has also been studied outside hybrid reasoning (Lee et al., 2026g,d). FrugalGPT (Chen et al., 2024a) and RouteLLM (Ong et al., 2025) route queries across separate models of different cost. Speculative decoding (Leviathan et al., 2023; Lee et al., 2026f; Lee and Eo, 2026; Lee et al., 2026c) coordinates token-level generation between a drafter and a verifier. Confident Adaptive Language Modeling (Schuster et al., 2022) exits early from decoding when confidence is high, and Adaptive Computation Time (Graves, 2017) learns halting decisions in recurrent networks.

7 Conclusion

We introduced DART, a training-free router for hybrid reasoning models that uses agreement among cheap NoThink drafts as answer-level evidence for when direct answering is sufficient. When drafts disagree, DART routes the query to Think mode and can allocate an entropy-predicted thinking budget, separating the reasoning trace from final-answer generation to avoid truncation artifacts. Across the main comparisons, DART matches or exceeds always-thinking across benchmarks while reducing thinking-token use by 32–73%, with the largest accuracy gains on code under execution-based equivalence. The analysis shows that draft agreement tracks difficulty across model families and scales (0.6B–32B), and it identifies false accepts as the main remaining error source, pointing to lightweight calibration or selective extra drafts as natural next steps.

Limitations

Scope and residual errors. DART targets open-ended generation with a large answer space, such as mathematical reasoning and code generation. Multiple-choice tasks fall outside this scope because a small option set makes chance agreement likely even when the shared answer is wrong. Table 14 in Appendix C reports that the point-biserial correlation between draft unanimity and AT correctness weakens sharply there, so we exclude them rather than retrofit a confidence threshold. Even within scope, draft agreement is a high-precision

signal rather than a correctness guarantee. When two NoThink drafts converge on the same wrong answer, DART accepts without invoking Think mode, and Section 5.2 identifies these false accepts as the main residual error source on MATH-500.

Equivalence functions. The routing decision depends on a domain-specific, pluggable answer extractor and answer-equivalence function, using answer normalization for math and sandboxed execution against each problem's reference tests for code. Each new domain needs its own equivalence check, neither too permissive nor too brittle, and a weak check can inflate false accepts or unnecessarily reduce accept rates. For code the check is not independent of the grader. Stage 1 executes both NoThink drafts against the same reference tests that later score the answer, so an accepted code query is correct by construction and code accept precision is 100% by definition rather than by measurement. To bound what this hides, we re-gated the stored HumanEval drafts on half of each problem's assertions and scored the returned program against the full suite, on the problems whose drafts the logs stored in full. Accept precision falls from a forced 100% to 92.5% at 8B, 97.9% at 14B and 98.1% at 32B, the band of the 90.8% we report on MATH-500, and accuracy falls by at most 3.7 points. The code gains in Table 1 are therefore gains under a gate that reads the evaluation tests.

Evaluation scope. The main results cover Qwen3 models on math and code together with DeepSeek-V3.2 on math, so behavior on other open-ended task families remains untested. Accuracy parity also depends on model capability, and Section 5 reports that DART trails AT on MATH-500 at the 0.6B–4B scales. AIME results in Appendix A are exploratory rather than confirmatory benchmark coverage.

Deployment assumptions. Stage 1 requires only generated text and is compatible with text-only APIs, whereas the entropy-budgeted Stage 2 additionally assumes access to token-level uncertainty signals and sufficient control over thinking budgets or stop behavior. Stage 2 also fits its entropy-budget map on a probe run that overlaps the math evaluation set, so the reported budgets are not a held-out estimate and new task families require refitting. Hosted APIs exposing only a subset of these controls can still run Stage 1 routing without the full entropy-budgeted gains. Table 2 and the MATH-500 row of Table 4 report total-token accounting over drafts, thinking, and answer tokens, but realized cost depends on provider pricing. Wall-clock latency is measured on a single A100-80GB configuration with parallel NoThink drafts, so absolute speedups may vary with serving stack, batching, API latency, and hardware.

Acknowledgements

This research was supported by Basic Science Research Program through the National Research Foundation of Korea(NRF) funded by the Ministry of Education(NRF-2021R1A6A1A03045425). This work was supported by Institute for Information & communications Technology Promotion(IITP) grant funded by the Korea government(MSIT). (RS-2024-00398115, Research on the reliability and coherence of outcomes produced by Generative AI). This research was supported by Culture, Sports and Tourism R&D Program, funded by the Ministry of Culture, Sports and Tourism, through the Korea Culture Technology Planning and Evaluation Institute, an affiliated institute of the Korea Creative Content Agency in 2026 (Project Name: Development of an AI Agent Integrating Korean Language Knowledge for Personalized Language Consultation Services, Project Number: RS-2026-25506607, Contribution Rate: 25%). This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) under the artificial intelligence star fellowship support program to nurture the best talents (IITP-2026-RS-2025-02304828) grant funded by the Korea government(MSIT).

References

Pranjal Aggarwal, Aman Madaan, Yiming Yang, and Mausam. 2023. Let's sample step by step: Adaptive-consistency for efficient reasoning and coding with LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12375–12396, Singapore. Association for Computational Linguistics.

Mohammad Ali Alomrani, Yingxue Zhang, Derek Li, Qianyi Sun, Soumyasundar Pal, Zhanguang Zhang, Yaochen Hu, Rohan Deepak Ajwani, Antonios Valkanas, Raika Karimi, Peng Cheng, Yunzhou Wang, Pengyi Liao, Hanrui Huang, Bin Wang, Jianye Hao, and Mark Coates. 2025. Reasoning on a budget: A survey of adaptive and controllable test-time compute in llms. *arXiv preprint arXiv:2507.02076*.

Anthropic. 2026. Building with extended thinking. Claude API documentation.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Miriam Ayer, H. D. Brunk, G. M. Ewing, W. T. Reid, and Edward Silverman. 1955. An empirical distribution function for sampling with incomplete information. *The Annals of Mathematical Statistics*, 26(4):641–647.

Lingjiao Chen, Matei Zaharia, and James Zou. 2024a. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*. Featured Certification.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Xinyun Chen, Renat Aksitov, Uri Alon, Jie Ren, Kefan Xiao, Pengcheng Yin, Sushant Prakash, Charles Sutton, Xuezhi Wang, and Denny Zhou. 2024b. Universal self-consistency for large language models. In *ICML 2024 Workshop on In-Context Learning*.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*.

DeepSeek-AI. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

DeepSeek-AI, Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, et al. 2025. Deepseek-v3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*.

Bowen Ding et al. 2025. Do thinking tokens help or trap? towards more efficient large reasoning model. *arXiv preprint arXiv:2506.23840*.

Sicheng Feng, Gongfan Fang, Xinyin Ma, and Xinchao Wang. 2025. Efficient reasoning models: A survey. *Transactions on Machine Learning Research*.

Google DeepMind. 2026. Gemma 4 model card. Google AI for Developers.

Alex Graves. 2017. Adaptive computation time for recurrent neural networks. *Preprint*, arXiv:1603.08983.

Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. 2024. OlympiadBench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3828–3850, Bangkok, Thailand. Association for Computational Linguistics.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the MATH dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*.

Maxwell Jia. 2024. AIME 2024. Hugging Face dataset.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. 2022. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*.

Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. 2023. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *The Eleventh International Conference on Learning Representations*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP '23, page 611–626, New York, NY, USA. Association for Computing Machinery.

Dongyub Jude Lee, Jungseob Lee, Chanjun Park, Hyeonseok Moon, and Heuiseok Lim. 2026a. Risk-controlled selective LLM answering by pricing label-free checks. *arXiv preprint arXiv:2609.37493*.

Jungseob Lee and Sugyeong Eo. 2026. CAST: Cost-aware speculative trees from one-pass block drafters. *arXiv preprint arXiv:2610.00321*.

Jungseob Lee, Sugyeong Eo, Seongtae Hong, Seungyoon Lee, Chanjun Park, Jaehyung Seo, and Heuiseok Lim. 2026b. Distilling directional verification. *arXiv preprint arXiv:2610.00997*.

Jungseob Lee, Seongtae Hong, Dongyub Jude Lee, Chanjun Park, Jaehyung Seo, Sugyeong Eo, and Heuiseok Lim. 2026c. Vision is not overhead: One-pass block drafting for lossless speculative decoding in vision-language models. *arXiv preprint arXiv:2609.00355*.

Jungseob Lee, Dongyub Jude Lee, Chanjun Park, Sugyeong Eo, and Heuiseok Lim. 2026d. Faster block-diffusion serving with distribution-free risk guarantees. *arXiv preprint arXiv:2609.33887*.

Jungseob Lee, Seungyoon Lee, Seongtae Hong, Minhyuk Kim, Chanjun Park, and Heuiseok Lim. 2026e. Beyond penalizing mistakes: Stabilizing efficiency training in large reasoning models via adaptive correct-only rewards. *arXiv preprint arXiv:2606.22716*.

Jungseob Lee, Chanjun Park, Sugyeong Eo, and Hyeonseok Moon. 2026f. Recovering off-policy supervision for speculative decoding. *arXiv preprint arXiv:2609.38795*.

Jungseob Lee, Chanjun Park, and Heuiseok Lim. 2026g. To isolate or to score? model-adaptive assessment for cost-efficient multi-agent RAG. *arXiv preprint arXiv:2606.25191*.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR.

Zheng Li, Qingxiu Dong, Jingyuan Ma, Di Zhang, Kai Jia, and Zhifang Sui. 2025. Selfbudgeter: Adaptive token allocation for efficient llm reasoning. *arXiv preprint arXiv:2505.11274*.

Guosheng Liang, Longguang Zhong, Ziyi Yang, and Xiaojun Quan. 2025. ThinkSwitcher: When to think hard, when to think fast. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 5185–5201, Suzhou, China. Association for Computational Linguistics.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In *The Twelfth International Conference on Learning Representations*.

Shangke Lyu, Linjuan Wu, Yuchen Yan, Xingyu Wu, Hao Li, Yongliang Shen, Peisheng Jiang, Weiming Lu, Jun Xiao, and Yueting Zhuang. 2025. Hierarchical budget policy optimization for adaptive reasoning. *Preprint*, arXiv:2507.15844.

Wenjie Ma, Jingxuan He, Charlie Snell, Tyler Griggs, Sewon Min, and Matei Zaharia. 2025. Reasoning models can be effective without thinking. *arXiv preprint arXiv:2504.09858*.

NVIDIA. 2026. Nemotron 3 nano omni: Efficient and open multimodal intelligence. *Preprint*, arXiv:2604.24954.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. 2025. RouteLLM: Learning to route LLMs from preference data. In *The Thirteenth International Conference on Learning Representations*.

OpenAI. 2026. Introducing GPT-5.5. OpenAI product release.

Test-Time Compute Organization. 2025. Aime 2025 - unified test-time scaling format. `https://huggingface.co/datasets/test-time-compute/aime_2025`.

Zhihong Pan, Kai Zhang, Yuze Zhao, and Yupeng Han. 2025. Route to reason: Adaptive routing for llm and reasoning strategy selection. *arXiv preprint arXiv:2505.19435*.

Qwen Team. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2024. GPQA: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*.

Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. 2022. Confident adaptive language modeling. In *Advances in Neural Information Processing Systems*.

Charlie Victor Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2025. Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning. In *The Thirteenth International Conference on Learning Representations*.

Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Na Zou, Hanjie Chen, and Xia Hu. 2025. Stop overthinking: A survey on efficient reasoning for large language models. *Transactions on Machine Learning Research*.

Shouren Wang, Wang Yang, Xianxuan Long, Qifan Wang, Vipin Chaudhary, and Xiaotian Han. 2025. Demystifying hybrid thinking: Can llms truly switch between think and no-think? *Preprint*, arXiv:2510.12680.

Xinglin Wang, Shaoxiong Feng, Yiwei Li, Peiwen Yuan, Yueqi Zhang, Chuyi Tan, Boyuan Pan, Yao Hu, and Kan Li. 2024a. Make every penny count: Difficulty-adaptive self-consistency for cost-efficient reasoning. *arXiv preprint arXiv:2408.13457*.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*.

Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. 2024b. Mmlu-pro: a more robust and challenging multi-task language understanding benchmark. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, Red Hook, NY, USA. Curran Associates Inc.

An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. 2024. Qwen2.5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*.

Tianyu Yu, Zefan Wang, Chongyi Wang, Fuwei Huang, Wenshuo Ma, Zhihui He, Tianchi Cai, Weize Chen, Yuxiang Huang, Yuanqian Zhao, et al. 2025. Minicpm-v 4.5: Cooking efficient mllms via architecture, data, and training recipe. *Preprint*, arXiv:2509.18154.

Jiajie Zhang, Nianyi Lin, Lei Hou, Ling Feng, and Juanzi Li. 2025a. AdaptThink: Reasoning models can learn when to think. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 3716–3730, Suzhou, China. Association for Computational Linguistics.

Xiaoyun Zhang, Jingqing Ruan, Xing Ma, Yawen Zhu, Haodong Zhao, Hao Li, Jiansong Chen, Ke Zeng, and Xunliang Cai. 2025b. When to continue thinking: Adaptive thinking mode switching for efficient reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 5808–5828, Suzhou, China. Association for Computational Linguistics.

A Additional Cross-Model and Small-Sample Results

AIME 2024/2025. Table 10 reports DART on AIME 2024 and 2025 for Qwen3-8B and DeepSeek-V3.2. We report these as exploratory evidence rather than confirmatory comparison. On Qwen3-8B, DART gains +6.7 on AIME 2024 and +3.4 on AIME 2025 over AT, with 32% and 29% thinking-token reductions. On DeepSeek-V3.2, DART matches AT on AIME 2024 and exceeds AT by 10 points, at 42% and 28% fewer thinking tokens.

Model	Benchmark	NT	AT	DART (vs AT)	Think↓
Qwen3-8B	AIME 2024	26.7	60.0	**66.7** (+6.7)	32%
	AIME 2025	26.7	53.3	**56.7** (+3.4)	29%
DeepSeek-V3.2	AIME 2024	66.7	70.0	70.0 (+0.0)	42%
	AIME 2025	53.3	40.0	**50.0** (+10.0)	28%

Table 10: DART on AIME 2024 / 2025 for Qwen3-8B and DeepSeek-V3.2. Exploratory results.

B Truncation Analysis and Budget Sweep

The one-phase AT accuracies reported in this appendix (Tables 11 and 12) use a diagnostic evaluation protocol designed for truncated one-phase outputs rather than the main-table protocol. Numbers are therefore not directly comparable to Table 8's DART accuracy. The trend, namely truncation at low budget and a plateau at high budget, is stable across evaluation protocols.

B	Acc	Trunc%	B	Acc	Trunc%
2K	13.5	86.0	22K	71.8	1.8
4K	40.0	49.2	24K	70.8	2.8
6K	55.0	28.7	26K	71.8	1.5
7K	59.0	23.5	28K	71.8	0.2
8K	61.0	19.5	30K	72.2	0.2
9K	65.7	14.5	32K	72.0	0.8
10K	66.0	12.5	36K	71.8	0.8
11K	67.5	10.5	40K	72.0	1.0
12K	68.8	9.2	44K	71.2	0.5
13K	68.8	6.8	48K	**72.2**	0.5
14K	69.8	5.5	52K	71.5	0.8
16K	69.5	5.0	56K	71.8	0.2
18K	70.0	3.8	60K	71.8	0.5
20K	71.5	2.2			
DART (two-stage, 1,743 mean thinking tokens): ***88.2***					

Table 11: One-phase AT accuracy under token budget on MATH-500 (Qwen3-8B).

C Hyperparameter Ablation (K) and Multiple-Choice Scope

Full K sweep at strict budget (MATH-500). Table 13 reports a sweep over $K \in \{1, 2, 3, 4\}$ on the first 200 MATH-500 problems, with every routed query capped at 4,096 thinking tokens. This is a separate run from the main experiments, which use all 500 problems at this scale and an entropy-predicted budget, and it redraws its drafts independently, so its accept rates and accuracies are comparable across K but not against Table 1. Accuracy peaks at K=3 (89.5%), with K=2 close behind (89.0%). K=1 degenerates to always-NT routing and underperforms (81.0%), while K=4 slightly regresses (88.5%) despite consuming 22% more thinking tokens than K=3. We therefore use K=2 as the practical efficiency point. Moving from K=2 to K=3 adds only a 0.5-point accuracy gain while increasing average thinking tokens from 772 to 939, and K=4 uses more thinking tokens without further accuracy. We adopt K=2 as the default in all other tables.

Multiple-choice task scope. The AT and NT columns of Table 14 separate two effects that Section 5 does not distinguish. THINK mode itself remains useful on all three benchmarks, with AT above NT at 92.3 against 86.1 on ARC-Challenge, 37.7 against 17.8 on MMLU-Pro, and 56.6 against 47.0 on GPQA-D. What collapses is the unanimity signal that Stage 1 reads. Its association with AT correctness is not significant on MMLU-Pro ($p = 0.49$) or GPQA-D ($p = 0.90$), and on ARC-Challenge it reaches significance only at a magnitude of 0.13. These runs keep the K=2 draft procedure of the main experiments and substitute an option-letter equivalence check for the math and code equivalence functions of §E.2. Together with the collision-probability argument given there, the near-zero association is why multiple-choice tasks stay outside the routing scope.

D Budget Ablation

Tables 15 and 16 sweep the thinking-budget cap for each query and the draft sampling temperature. Accuracy plateaus once the budget exceeds the entropy-predicted average (about 2.8K on MATH-500), with no benefit from larger caps. Below the predicted average, accuracy degrades sharply, supporting the entropy-to-budget mapping in Eq. 4 and Figure 4. The last two rows of Table 15 instead isolate Stage 2. Both serve the same Stage 1 accepts, and the control reassigns the predicted budgets of the 110 queries Stage 1 routes to THINK mode by resampling those same budgets with replacement,

Budget	Acc	Trunc%
2K	1.9	98.7
4K	14.6	79.2
8K	36.7	45.8
16K	53.8	16.5
32K	58.9	2.3
49K	58.1	3.8

Table 12: One-phase AT on OlympiadBench (Qwen3-8B).

K	Accuracy (%)	NT-route rate	Avg. think tok.
1	81.0	1.000	0
2	89.0	0.810	772
3	**89.5**	0.760	939
4	88.5	0.705	1,149

Table 13: Accuracy and routing behavior as a function of K on Qwen3-8B over the first 200 MATH-500 problems, with a strict 4,096-token budget cap for each query. Separate run from Table 1.

so the budget distribution is matched and only the pairing between draft entropy and query is destroyed. We report its mean and standard deviation over 100 seeds. The entropy-conditioned schedule covers the realized thinking-token demand of all 110 routed queries, while the shuffled schedule under-budgets part of the set.

Effect of T (sampling temperature). Table 16 varies the draft sampling temperature on DeepSeek-V3.2 over a 100-problem MATH-500 subset, so its absolute values are not comparable to the DeepSeek-V3.2 row of Table 1 and only the ordering across T is meaningful. NT% and SC% are the accuracies of fixed NoThink generation and of Stage 1 routing, while Accept%, Prec.%, and r_{pb} are the unanimous-accept rate, the accept precision, and the point-biserial correlation defined in Section 3. The accept rate is lowest at T=0.8 (85.0), which is also where accept precision is highest (100.0), so a higher draft temperature trades accepted queries for precision on the queries Stage 1 still keeps. Each column peaks at a different temperature, with SC% highest at T=0.7 (96.0), Prec.% and r_{pb} at T=0.8, and NT% at T=0.9 (93.0), while r_{pb} stays above 0.69 at every setting, so the agreement signal Stage 1 depends on survives the whole sweep. We keep T=0.7, the hosted-API default used for these runs.

Benchmark	AT	NT	r_{pb}	p
ARC-C	**92.3**	86.1	0.13	<0.001
MMLU-Pro	**37.7**	17.8	0.06	0.49
GPQA-D	**56.6**	47.0	−0.009	0.90

Table 14: Multiple-choice benchmarks on Qwen3-8B. r_{pb} is the point-biserial correlation between draft unanimity and AT correctness.

B	Acc	B	Acc
1K	69.25	12K	71.75
2K	69.75	14K	**73.00**
3K	70.75	16K	72.25
4K	70.00	20K	70.00
5K	70.50	24K	72.25
6K	71.25	28K	71.50
8K	70.00	32K	72.25
10K	71.00	49K	71.25
Strategy comparison under the main evaluation protocol			
Shuffled budget (100 seeds)			87.3 ± 0.3
DART (entropy-conditioned)			**88.2**

Table 15: Budget ablation on MATH-500 (Qwen3-8B), string-extract grader on 400 problems. The last two rows use the main evaluation protocol on all 500 problems and are not comparable to the sweep above them.

E Implementation Details

Qwen3-8B. Local serving uses vLLM (Kwon et al., 2023); the installed version is 0.16.0 with reasoning-output support enabled. Drafts use T=0.6, top-p=0.95, `enable_thinking=false`, and a maximum of 1024 tokens following Qwen3 chat-template defaults. Escalation uses T=0.0 (greedy), `enable_thinking=true`, and a 16,384-token thinking maximum, the same cap the AT baseline receives. The 4,096-token cap appears only in the strict-budget sweep of §C. Hardware is NVIDIA A100-80GB.

DeepSeek-V3.2. We queried the hosted DeepSeek API before the 2026-04-24 alias transition, when `deepseek-reasoner` and `deepseek-chat` mapped to DeepSeek-V3.2 thinking and non-thinking modes. Drafts use the `deepseek-chat` endpoint at API defaults.

Artifact access and terms. Model, benchmark, API, and software artifacts were used only for research evaluation under the licenses or access terms stated by their original providers. We cite the original artifact creators where the artifacts are introduced, including Section 3 and Appendix E. We do not redistribute third-party model weights, benchmark data, hosted API outputs as

T	NT%	SC%	Accept%	Prec.%	r_{pb}
0.5	91.0	95.0	88.0	98.9	0.744
0.6	91.0	92.0	89.0	98.9	0.783
0.7	89.0	**96.0**	90.0	97.8	0.842
0.8	86.0	93.0	85.0	**100.0**	**0.960**
0.9	93.0	94.0	90.0	98.9	0.692

Table 16: Temperature sensitivity on DeepSeek-V3.2 MATH-500. T=0.7 is the default.

a standalone dataset, or modified third-party artifacts. Hosted-model experiments were conducted through provider APIs under their service terms.

Answer extraction. Math answers are extracted from the final `\boxed{...}` span on MATH-500 and OlympiadBench, code answers are taken as the generated program and executed against the reference tests described in §E.2, and multiple-choice answers are read from the emitted option letter.

E.1 Evaluation Protocol Details

Math accuracies in the main tables use the Qwen2.5-Math semantic-equivalence grader (Yang et al., 2024) (`math_equal`) applied to full responses. Appendix budget-sweep tables (§B) instead use a string-extract grader for truncation diagnosis, since one-phase responses may end mid-LaTeX and confuse the semantic grader. These numbers are therefore not directly comparable to main results. Code accuracies use execution-based equivalence against each problem's reference tests, the same tests Stage 1 reads when routing, on the full HumanEval and MBPP sets for every model. Rows of Tables 1 and 5 are single-run and cover the full benchmark sizes of Section 3, except that Qwen3-32B MATH-500 covers 348 of 500 problems and OlympiadBench covers 200 of 572 problems at Qwen3-14B and Qwen3-32B. Supervised-router rows in Table 2 use 5-fold cross-validation on benchmark-matched subsets.

E.2 Pluggable Equivalence Functions

The routing decision hinges on whether two draft answers "agree." For mathematical reasoning, we normalize extracted answers by stripping LaTeX, canonicalizing numbers, and lowercasing strings. For code generation, string comparison fails because functionally identical programs rarely match character by character. We instead execute both drafts against each problem's reference test suite in a sandboxed environment and define agreement as both drafts passing all of its tests. No tests are held out, so the suite Stage 1 reads is the suite that scores the final answer, and an accepted code query passes by construction. The equivalence module is the *only* domain-specific component. Extending DART to a new domain requires only implementing the appropriate eq, and the routing mechanism remains unchanged.

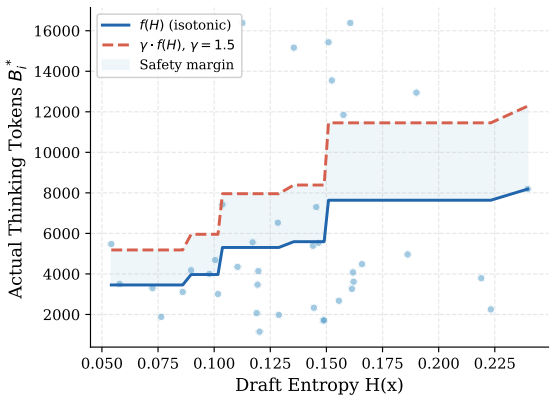


Figure 4: Draft entropy vs. actual thinking tokens on MATH-500 calibration data. The solid line is the isotonic mapping $f(H)$. The dashed line is the safety-margin schedule $\gamma \cdot f(H)$ with γ=1.5.

F Entropy–Budget Calibration

Figure 4 plots the isotonic-regression mapping f from draft entropy $H(q)$ to predicted thinking budget $\hat{B}(q)$ used in Stage 2 (Eq. 4). We fit the mapping label-free on a 100-problem probe run of disagreement-flagged math queries, 71 of which also appear in the evaluation set. The mapping is monotone non-decreasing by construction. Queries with diffuse draft distributions, and therefore higher entropy, receive larger thinking budgets, while low-entropy disagreement queries receive tight budgets and recover quickly. The safety margin γ=1.5 shifts the curve upward so the predicted budget exceeds the observed thinking-token usage at the matched entropy.

G Self-Verification as an Alternative Router

A natural alternative to draft unanimity is to ask the model itself whether its NT draft is correct. We evaluated this on a MATH-500 pilot, prompting Qwen3-8B in no-think mode to judge its own draft as *yes*/*no*/*unclear*, and routing to AT whenever the verdict was not *yes*. Using the true correctness of the draft as ground truth, verifier precision is 0.638 (of drafts labeled *yes*, 37/58 are actually correct)

Path	Latency (s)	Share
AT (full thinking)	67.6	—
DART (overall mean)	29.5	100%
DART (no-think exit)	14.6	78%

Table 17: Wall-clock latency on each MATH-500 query (Qwen3-8B, A100-80GB). DART overall is the weighted mean across the two paths. No-think exit corresponds to Stage 1 unanimity accept.

and recall is 0.841 (of correct drafts, $37/44$ are accepted). Translated into routing behavior, the verifier *detects* only 22.2% of wrong NT drafts (detection rate, true-negative share among wrong drafts) while *preserving* 84.1% of correct ones (preservation rate). Among the wrong drafts it does route to AT, only 3 are actually recovered, leaving routing accuracy below DART's draft-unanimity rule. Self-verification thus combines low sensitivity to errors with collateral loss of correct answers. We did not pursue it further in the main pipeline.

H Wall-clock Latency Breakdown

Table 17 gives the measurement behind the latency reduction reported in Section 4.3. Timings are per-query wall clock for Qwen3-8B served locally on a single A100-80GB node under the sampling settings of §E, measured on MATH-500. The $K=2$ NoThink drafts are generated in parallel, so a Stage 1 accept returns after one draft pass and never enters Think mode. The reported DART mean is the share-weighted combination of that accept path and the Think path, so it moves with the accept rate rather than with the thinking budget. These numbers describe one serving configuration and are not a portable speedup, since batching, serving stack, and hardware all change the absolute values.